

Cours 07 — Les bonnes pratiques

Module 7/8 — Formation Git & GitHub

Objectifs

À la fin de ce module, vous saurez : - écrire des messages de commit utiles au format **Conventional Commits** ; - rédiger un `.gitignore` correct et retirer du suivi un fichier committé par erreur ; - choisir un **modèle de branches** adapté à votre projet ; - versionner avec des **tags**, publier des **releases**, et raisonner en **SemVer**.

Théorie

Conventional Commits : un message qui parle aux humains et aux outils

Un message de commit sert à celui qui fera `git log` dans six mois — souvent vous-même. La convention **Conventional Commits** standardise le format pour qu'il soit lisible **et** exploitable par des outils (génération de changelog, calcul automatique de version) :

```
<type>(<portée facultative>): <description à l'impératif>
```

```
feat(ordres): ajoute le support des ordres stop-loss
```

```
fix(export): corrige l'encodage UTF-8 du CSV
```

```
docs: précise les prérequis dans le README
```

Les types courants :

Type	Pour
feat	une nouvelle fonctionnalité
fix	une correction de bug
docs	de la documentation seulement
refactor	une réécriture sans changer le comportement
test	ajout ou correction de tests
chore	tâche d'entretien (dépendances, config...)
perf / ci	performance / configuration d'intégration continue

Le **corps** du message (facultatif, après une ligne vide) explique le **pourquoi**. Un **!** après le type — `feat!` : — signale un **changement cassant** (voir SemVer plus bas). La description se met à l'impératif présent : « ajoute », pas « ajouté » ni « ajout ».

`.gitignore` : décider ce que Git ignore

Un `.gitignore` liste les fichiers que Git ne doit **jamais** suivre : artefacts de compilation, dépendances, secrets locaux, réglages d'éditeur. À créer **dès le premier commit**. Pour un projet Python typique :

```
__pycache__/  
*.py[cod]  
.venv/  
.idea/  
.vscode/  
*.log  
.env          # secrets locaux : JAMAIS dans Git  
dist/  
build/
```

Piège classique : `.gitignore` n'agit que sur les fichiers **pas encore suivis**. Un fichier **déjà committé** continue d'être suivi même si vous l'ajoutez au `.gitignore` — il faut d'abord le retirer du suivi (`git rm --cached`, voir plus bas).

Modèles de branches : combien de couloirs ?

Modèle	Principe	Pour qui
GitHub Flow	main + branches courtes + PR ; déploiement continu	La plupart des projets
Git Flow	main, develop, release/*, hotfix/*	Versions packagées, cycles longs
Trunk-based	Branches de quelques heures + feature flags	Équipes matures, CI solide

Par défaut : **GitHub Flow**, et rien d'autre tant que le besoin ne se fait pas sentir. Simple, il suffit à l'immense majorité des projets — dont tous ceux de cette formation. On ne complexifie le modèle de branches que quand une vraie contrainte l'exige (livraisons versionnées, support de plusieurs versions en parallèle).

Tags, releases et SemVer : nommer les versions

Un **tag** fige un commit sous un nom lisible (`v1.4.0`) : un signet permanent dans l'historique. Une **release** GitHub s'appuie sur un tag et y ajoute des notes de version et, éventuellement, des binaires téléchargeables.

SemVer (*Semantic Versioning*) donne un sens à chaque partie du numéro MAJEUR.MINEUR.CORRECTIF :

- **MAJEUR** — un changement **incompatible** (l'ancien usage casse). C'est le feat ! de tout à l'heure.
- **MINEUR** — une **nouvelle fonctionnalité** rétrocompatible (rien ne casse).
- **CORRECTIF** — une **correction de bug** rétrocompatible.

Ainsi, en voyant passer `1.4.0` → `2.0.0`, un utilisateur sait qu'il doit s'attendre à devoir adapter son code ; `1.4.0` → `1.5.0` lui promet que tout continue de marcher.

Commandes commentées

Taguer une version — `git tag`

```
git tag -a v1.0.0 -m "Première version stable"    # tag ANNOTÉ (auteur, date, message)
git tag                                           # lister tous les tags
git show v1.0.0                                  # voir le tag et le commit qu'il désigne
```

Préférez le tag **annoté** (`-a`) au tag léger : il porte un auteur, une date et un message, comme un mini-commit. Le tag léger (`git tag v1.0.0` sans `-a`) n'est qu'une étiquette nue, réservez-le aux repères temporaires.

Pousser les tags — `git push`

```
git push origin v1.0.0                          # pousser UN tag précis
git push --tags                                  # pousser TOUS les tags locaux d'un coup
```

Attention : les tags **ne partent pas** avec un `git push` normal — ils sont ignorés par défaut. Il faut les pousser explicitement, sinon votre `v1.0.0` reste local et la release GitHub ne pourra pas s'y accrocher.

Publier une release — `gh release`

```
gh release create v1.0.0 --title "v1.0.0" --notes "Première version stable"
```

La CLI gh (cours 08) crée la release GitHub directement, sans passer par le navigateur. Elle crée le tag si besoin, y attache les notes, et la version apparaît dans l'onglet **Releases** du dépôt.

Oublier un fichier committé par erreur — `git rm --cached`

```
git rm --cached .env          # retire .env du SUIVI, le laisse sur le disque
echo ".env" >> .gitignore     # et on l'ignore désormais
git commit -m "chore: retire .env du suivi"
```

`git rm --cached` dé-suit un fichier **sans le supprimer** de votre disque — exactement ce qu'il faut quand on a committé par mégarde un fichier local (secrets, config perso). **Mais** : le contenu reste dans l'**historique** des commits précédents. Si c'était un secret (clé d'API, mot de passe), le retirer du suivi ne suffit **pas** — il faut le considérer comme **compromis** et le **révoquer/régénérer** immédiatement.

Exercices pratiques

1. **Messages.** Réécrivez ces trois messages au format Conventional Commits : « corrigé bug », « ajout fichier », « maj readme ».
2. **.gitignore.** Ajoutez le `.gitignore` Python ci-dessus à votre dépôt, créez un fichier `.env`, et vérifiez qu'il **n'apparaît plus** dans `git status`.
3. **Fichier à oublier.** Committez volontairement un `secret.txt`, puis retirez-le du suivi avec `git rm --cached` **sans** le supprimer du disque. Vérifiez qu'il est bien encore là.
4. **Release.** Taguez `v0.1.0` (annoté) sur votre dépôt, poussez le tag, et créez la release correspondante sur GitHub avec des notes de version.

Solutions des exercices

Solution 1 — Messages

```
« corrigé bug »   → fix: corrige le calcul du prix moyen
« ajout fichier » → feat: ajoute le module d'export CSV
« maj readme »   → docs: met à jour les prérequis du README
```

Chaque message : un **type** (fix/feat/docs), puis une description à l'**impératif** qui dit **quoi** précisément. « corrigé bug » ne dit rien à celui qui relira le `git log` dans six mois ; « fix: corrige le calcul du prix moyen » situe immédiatement le changement.

Solution 2 — `.gitignore`

```
# créer le .gitignore avec le contenu Python du cours
git add .gitignore && git commit -m "chore: ajoute le .gitignore Python"
```

```
echo "SECRET=abc123" > .env          # créer un .env local
git status
```

```
# → .env N'APPARAÎT PAS dans les fichiers non suivis : Git l'ignore, comme voulu
```

Comme `.env` figure dans `.gitignore` **et** n'a jamais été committé, Git ne le voit plus du tout. C'est le comportement recherché : les secrets locaux restent locaux.

Solution 3 — Fichier à oublier

```
echo "clé = 42" > secret.txt
git add secret.txt && git commit -m "oops: secret committé par erreur"
```

```
git rm --cached secret.txt          # retire du suivi, garde le fichier sur le
disque
```

```
ls secret.txt                      # → secret.txt : il est TOUJOURS là
```

```
echo "secret.txt" >> .gitignore
```

```
git commit -m "chore: retire secret.txt du suivi"
```

```
git status # secret.txt n'est plus suivi ni signalé
```

Rappel critique : le contenu de `secret.txt` **reste** dans le commit « oups » de l'historique. Pour un vrai secret, il faut le **révoquer** — le retirer du suivi ne le fait pas disparaître du passé.

Solution 4 — Release

```
git tag -a v0.1.0 -m "Première version : gestionnaire d'ordres de base"
git push origin v0.1.0 # le tag ne serait pas parti avec un push normal

gh release create v0.1.0 --title "v0.1.0" \
  --notes "Première version publique. Ordres market + export CSV."
```

Sans gh, faites-le à la main : onglet **Releases** → **Draft a new release**, choisissez le tag `v0.1.0`, rédigez les notes, **Publish release**. La version apparaît alors dans l'onglet **Releases**, téléchargeable.

Points clés à retenir

- Un message de commit s'écrit pour celui qui fera `git log` dans six mois — souvent **vous**. Type + description à l'impératif : `feat: ..., fix: ..., docs: ...`
- `.gitignore` **dès le premier commit** ; il n'agit que sur les fichiers **pas encore suivis**.
- Un secret committé est un secret **compromis** : `git rm --cached` ne suffit pas, **révoquez-le**.
- **GitHub Flow** par défaut ; on ne complexifie le modèle de branches que si une contrainte réelle l'impose.
- Tag **annoté** + release + **SemVer** = des versions traçables, et un numéro qui prévient l'utilisateur de ce qui casse (MAJEUR) ou pas.